

Code Explanation

DLT Pipeline Test Cases Explanation

This document provides a step-by-step breakdown of the test cases for a DLT (Data Lake Transformation) pipeline. The test cases are written using PySpark and Pytest.

The code is designed to test various aspects of the DLT pipeline, including data cleaning, joining, and aggregation. The test cases utilize sample customer and order data to verify the correctness of the pipeline.

Overview of the Code

The test code is divided into several sections:

- Fixtures for creating a Spark session and sample customer and order data
- Test cases for cleaning customer and order data
- Test cases for joining customer and order data
- Test cases for aggregating customer spending

Step 1: Creating a Spark Session Fixture

The first step is to create a Spark session fixture using Pytest's `@pytest.fixture` decorator. This fixture is used to create a Spark session with the necessary configurations for the test cases.

```
@pytest.fixture(scope="session")
def spark():
    """Create a Spark session for testing."""
    return (
        SparkSession.builder
            .appName("CustomerOrderProcessingTest")
            .master("local[*]")
            .config("spark.sql.warehouse.dir", "file:///tmp/spark-wareh
use")
            .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSess
ionExtension")
            .config("spark.sql.catalog.spark_catalog", "org.apache.spark
.sql.delta.catalog.DeltaCatalog")
            .getOrCreate()
    )
```

Step 2: Creating Sample Customer and Order Data Fixtures

The next step is to create sample customer and order data fixtures using PySpark. These fixtures are used to test the various aspects of the DLT pipeline.

```
@pytest.fixture
def sample_customer_data(spark):
    """Create sample customer data for testing."""
    schema = StructType([
        StructField("CustId", StringType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])
    data = [
```

```

        ("C001", "John Doe", "john.doe@example.com", "North"),
        ("C002", "Jane Smith", "jane.smith@example.com", "South"),
        ("C003", "Bob Johnson", "bob.johnson@example.com", "East"),
        ("C004", "Alice Brown", "alice.brown@example.com", "West"),
        ("C005", None, "invalid@example.com", "North"), # Null name
        ("C001", "John Doe", "john.doe@example.com", "North") # Duplicate
    ]

    return spark.createDataFrame(data, schema)

@pytest.fixture
def sample_order_data(spark):
    """Create sample order data for testing."""
    schema = StructType([
        StructField("OrderId", StringType(), True),
        StructField("ItemName", StringType(), True),
        StructField("PricePerUnit", DoubleType(), True),
        StructField("Qty", IntegerType(), True),
        StructField("Date", DateType(), True),
        StructField("CustId", StringType(), True)
    ])

    data = [
        ("0001", "Laptop", 1200.0, 1, datetime.date(2023, 1, 15), "C001"),
        ("0002", "Mouse", 25.0, 2, datetime.date(2023, 1, 20), "C002"),
        ("0003", "Keyboard", 50.0, 1, datetime.date(2023, 1, 25), "C003"),
        ("0004", "Monitor", 300.0, 2, datetime.date(2023, 1, 30), "C001"),
        ("0005", "Headphones", 100.0, None, datetime.date(2023, 2, 5), "C002"), # Null quantity
        ("0001", "Laptop", 1200.0, 1, datetime.date(2023, 1, 15), "C001") # Duplicate
    ]

    return spark.createDataFrame(data, schema)

```

Step 3: Testing Data Cleaning for Customer Data

The `test\_clean\_customer\_data` function tests the cleaning of customer data by removing null values and duplicates.

```

def test_clean_customer_data(spark, sample_customer_data):
    """Test cleaning of customer data (removing nulls and duplicates)."""
    cleaned_df = (
        sample_customer_data
        .filter(
            (F.col("CustId").isNotNull()) &
            (F.col("Name").isNotNull()) &
            (F.col("EmailId").isNotNull()) &
            (F.col("Region").isNotNull())
        )
        .dropDuplicates(["CustId"])
    )

```

```

    )

    assert cleaned_df.count() == 4 # Should have 4 valid unique customers
    assert "C005" not in [row.CustId for row in cleaned_df.collect()]
    # Null name record should be removed

```

Step 4: Testing Data Cleaning and Total Amount Calculation for Order Data

The `test\_clean\_order\_data\_with\_total\_amount` function tests the cleaning of order data by removing null values and duplicates, and calculating the total amount.

```

def test_clean_order_data_with_total_amount(spark, sample_order_data):
    """Test cleaning of order data and calculation of TotalAmount."""
    cleaned_df = (
        sample_order_data
        .filter(
            (F.col("OrderId").isNotNull()) &
            (F.col("ItemName").isNotNull()) &
            (F.col("PricePerUnit").isNotNull()) &
            (F.col("Qty").isNotNull()) &
            (F.col("Date").isNotNull()) &
            (F.col("CustId").isNotNull())
        )
        .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
        .dropDuplicates(["OrderId"])
    )

    assert cleaned_df.count() == 4 # Should have 4 valid unique orders
    assert "0005" not in [row.OrderId for row in cleaned_df.collect()]
    # Null qty record should be removed

    order_o002 = cleaned_df.filter(F.col("OrderId") == "0002").first()
    assert order_o002.TotalAmount == 50.0 # 25.0 * 2 = 50.0

```

Step 5: Testing Joining Customer and Order Data

The `test\_join\_customer\_and\_order` function tests the joining of customer and order data.

```

def test_join_customer_and_order(spark, sample_customer_data, sample_order_data):
    """Test joining customer and order data."""
    clean_customer = (
        sample_customer_data
        .filter(
            (F.col("CustId").isNotNull()) &
            (F.col("Name").isNotNull()) &
            (F.col("EmailId").isNotNull()) &
            (F.col("Region").isNotNull())
        )
    )

```

```

        .dropDuplicates(["CustId"])
    )
    clean_order = (
        sample_order_data
        .filter(
            (F.col("OrderId").isNotNull()) &
            (F.col("ItemName").isNotNull()) &
            (F.col("PricePerUnit").isNotNull()) &
            (F.col("Qty").isNotNull()) &
            (F.col("Date").isNotNull()) &
            (F.col("CustId").isNotNull())
        )
        .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
        .dropDuplicates(["OrderId"])
    )

    joined_df = (
        clean_customer
        .join(
            clean_order,
            "CustId",
            "inner"
        )
        .select(
            "CustId", "Name", "EmailId", "Region", "OrderId",
            "ItemName", "PricePerUnit", "Qty", "Date", "TotalAmount"
        )
    )

    assert joined_df.count() == 4 # Should have 4 valid joined records

    john_orders = joined_df.filter(F.col("Name") == "John Doe").count()
    assert john_orders == 2 # John Doe has 2 orders (0001 and 0004)

```

Step 6: Testing Aggregation of Customer Spending

The `test\_customer\_aggregate\_spend` function tests the aggregation of customer spending.

```

def test_customer_aggregate_spend(spark, sample_customer_data, sample_order_data):
    """Test aggregation of customer spending."""
    clean_customer = (
        sample_customer_data
        .filter(
            (F.col("CustId").isNotNull()) &
            (F.col("Name").isNotNull()) &
            (F.col("EmailId").isNotNull()) &
            (F.col("Region").isNotNull())
        )
        .dropDuplicates(["CustId"])
    )

    clean_order = (

```

```

        sample_order_data
        .filter(
            (F.col("OrderId").isNotNull()) &
            (F.col("ItemName").isNotNull()) &
            (F.col("PricePerUnit").isNotNull()) &
            (F.col("Qty").isNotNull()) &
            (F.col("Date").isNotNull()) &
            (F.col("CustId").isNotNull())
        )
        .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
        .dropDuplicates(["OrderId"])
    )

    joined_df = (
        clean_customer
        .join(
            clean_order,
            "CustId",
            "inner"
        )
        .select(
            "CustId", "Name", "EmailId", "Region", "OrderId",
            "ItemName", "PricePerUnit", "Qty", "Date", "TotalAmount"
        )
        .withColumn("IsActive", F.lit(True))
    )

    agg_df = (
        joined_df
        .filter(F.col("IsActive") == True)
        .groupBy("Name", "Date")
        .agg(F.sum("TotalAmount").alias("TotalAmount"))
    )

    assert agg_df.count() == 4 # Should have 4 aggregated records (
one per customer-date combination)

    john_jan_15 = agg_df.filter((F.col("Name") == "John Doe") & (F.col("Date") == datetime.date(2023, 1, 15))).first()
    assert john_jan_15.TotalAmount == 1200.0 # John's Jan 15 order
was $1200

```